

**EXPLAINABLE AI AND HUMAN-IN-THE-LOOP FEEDBACK
FOR DOCUMENT QUESTION ANSWERING: A GENERATIVE
APPROACH USING BART ENCODER-DECODER WITH
CROSS-ATTENTION AND LIME**

Mithula Minsandha Pathirana

(IT22335050)

Dissertation submitted in partial fulfillment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specializing in Data Science

Department of Data Science
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: _____ Date: _____

The above candidate has carried out research for the Bachelor's Degree Dissertation under my supervision.

Signature of the Supervisor: _____ Date: _____

Name of the Supervisor: Ms. Samadhi Rathnayake

ABSTRACT

The growing adoption of artificial intelligence in document processing has made transparency and interpretability more important than ever, especially in automated question answering systems. This dissertation presents the design, development, and evaluation of an Explainable AI engine and a Human-in-the-Loop feedback system, both of which form part of a larger modular multi-agent document AI platform.

The Explainable AI component uses a BART encoder-decoder architecture, specifically the vblagoje/bart_lfqa model, to generate natural language answers from financial documents. This replaces traditional extractive approaches that can only pull out exact text spans. Explainability is delivered through two complementary methods: cross-attention extraction from the decoder layers, which provides token-level importance scores, and LIME (Local Interpretable Model-agnostic Explanations), which uses input perturbation to identify which words most influenced the model's output.

Questions are routed through a three-stage Waterfall Pipeline. The first stage handles conversational queries. The second applies rule-based handlers for yes/no, list, and comparison questions. The third falls back to the full BART model for complex factual queries. Supporting components include spaCy Named Entity Recognition (NER), question decomposition for comparative queries, table reconstruction for flattened OCR text, and an answer validator that catches hallucinated numbers.

The Human-in-the-Loop module allows domain experts to rate answer correctness, score explanation quality across three dimensions, and suggest knowledge updates. All feedback is persisted to PostgreSQL for ongoing improvement. A lazy XAI loading pattern separates fast answer delivery from computationally expensive explanation generation, reducing perceived latency significantly.

Evaluation on real financial documents showed the generative approach produces more natural and contextually appropriate answers than extractive baselines. The XAI visualizations provided meaningful insight into the model's reasoning, and the HITL feedback loop established a clear path for iterative quality improvement through human oversight.

Keywords: Explainable AI, Human-in-the-Loop, BART, Cross-Attention, LIME, Document Question Answering, Generative QA, Financial Document Processing

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. Samadhi Rathnayake, for her guidance, continuous support, and constructive feedback throughout this research. Her expertise and encouragement played a central role in shaping both the direction and the quality of this work.

I extend my appreciation to the Department of Data Science at the Sri Lanka Institute of Information Technology for providing the academic environment and resources necessary to conduct this research.

I am also grateful to my fellow team members who contributed to the broader modular multi-agent document AI platform. Their work made it possible to integrate the XAI and HITL components into a complete and coherent system architecture.

I want to thank the open-source communities behind PyTorch, Hugging Face Transformers, LIME, spaCy, FastAPI, and Next.js. These tools and frameworks formed the technical foundation of everything built in this project.

Finally, I express my heartfelt thanks to my family and friends for their unwavering support and encouragement throughout my academic journey.

TABLE OF CONTENTS

DECLARATION	ii
ABSTRACT.....	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS.....	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: INTRODUCTION	1
1.1 Background and Literature Survey	1
1.1.1 Document Question Answering Systems.....	1
1.1.2 Extractive vs. Generative Question Answering	2
1.1.3 Explainable AI in Natural Language Processing	2
1.1.4 Human-in-the-Loop Machine Learning	3
1.1.5 Financial Document Processing.....	4
1.2 Research Gap	4
1.3 Research Problem	5
1.4 Research Objectives	6
CHAPTER 2: METHODOLOGY	8
2.1 System Architecture	8
2.1.1 Microservice Design	8
2.1.2 XAI-HITL Service Architecture	8
2.2 Generative Q&A Engine	9
2.2.1 BART Encoder-Decoder Model	9
2.2.2 Input Formatting and Tokenisation	10
2.2.3 Beam Search Generation.....	10
2.2.4 Confidence Estimation	11
2.3 Explainable AI Pipeline	11
2.3.1 Cross-Attention Extraction	11
2.3.2 LIME Explanation with Sequence Probability	12
2.3.3 Combined Score Calculation	13
2.3.4 Highlighted Span Generation.....	13
2.3.5 Derivation Tree Construction.....	14

2.4 Waterfall Question Processing Pipeline.....	14
2.4.1 Stage 1: Conversational Handling.....	15
2.4.2 Stage 2: Rule-Based Processing.....	15
2.4.2.1 Yes/No Question Handler	16
2.4.2.2 List Question Handler with spaCy NER	16
2.4.2.3 Comparison Question Decomposition	17
2.4.3 Stage 3: Model-Generated Inference	17
2.5 Financial Text Pre-Processing	17
2.5.1 Table Reconstruction for Flattened OCR Text	17
2.5.2 Entity Anchor Search	18
2.5.3 Answer Validation and Hallucination Prevention.....	18
2.6 Human-in-the-Loop Feedback System	19
2.6.1 Feedback Data Model	19
2.6.2 Multi-Dimensional Rating System.....	20
2.6.3 Persistence and History Management	20
2.6.4 HITL-AHMN Integration Bridge.....	20
2.7 Lazy XAI Loading Pattern	21
2.8 Frontend Implementation	22
2.8.1 Chat Interface for Document Q&A.....	22
2.8.2 XAI Explanation Visualisation	22
2.8.3 Expert Review Workflow	22
2.9 Testing and Implementation.....	23
2.9.1 Unit Testing Strategy	23
2.9.2 Docker Deployment	23
2.9.3 API Endpoint Design	24
CHAPTER 3: RESULTS AND DISCUSSION.....	25
3.1 Results	25
3.1.1 Generative vs. Extractive Answer Quality	25
3.1.2 Cross-Attention Token Importance Analysis.....	26
3.1.3 LIME Word-Level Explanation Quality	26
3.1.4 Waterfall Pipeline Routing Accuracy	27
3.1.5 Financial Text Processing Effectiveness.....	27
3.1.6 HITL Feedback Collection and Statistics	28
3.2 Research Findings	28
3.2.1 Generative Approach Advantages.....	28

3.2.2 Cross-Attention as an XAI Mechanism	29
3.2.3 Lazy XAI Pattern Effectiveness.....	29
3.2.4 HITL Feedback Quality	29
3.3 Discussion	30
CHAPTER 4: CONCLUSION.....	32
4.1 Summary of Contributions.....	32
4.2 Limitations	33
4.3 Future Work	34
REFERENCES.....	35

LIST OF FIGURES

Figure	Title	Page
Figure 2.1	Overall System Architecture of the Multi-Agent Platform	13
Figure 2.2	XAI-HITL Service Internal Architecture	15
Figure 2.3	BART Encoder-Decoder Architecture for Q&A	18
Figure 2.4	Cross-Attention Extraction from Decoder to Encoder	22
Figure 2.5	LIME Perturbation Process for Generative Models	24
Figure 2.6	Combined Score Visualization	26
Figure 2.7	Waterfall Question Processing Pipeline	28
Figure 2.8	Question Decomposition for Comparison Queries	33
Figure 2.9	Table Reconstruction Pipeline for Flattened Text	35
Figure 2.10	HITL Feedback Data Flow	39
Figure 2.11	Lazy XAI Loading Sequence Diagram	42
Figure 2.12	XAI Chat Interface Screenshot	43
Figure 2.13	XAI Explanation Page with Feature Importance	44
Figure 2.14	HITL Expert Review Interface	45
Figure 2.15	Docker Container Architecture	47
Figure 3.1	Comparison of Answer Quality: Generative vs. Extractive	50
Figure 3.2	Cross-Attention Heatmap for Sample Financial Query	51
Figure 3.3	LIME Word Weights for Financial Document Q&A	52
Figure 3.4	Waterfall Pipeline Question Type Distribution	53
Figure 3.5	HITL Feedback Distribution by Type	55

LIST OF TABLES

Table	Title	Page
Table 2.1	Technology Stack for XAI-HITL Component	16
Table 2.2	BART Model Configuration Parameters	19
Table 2.3	Highlight Type Classification Thresholds	27
Table 2.4	Question Type Detection Patterns	29
Table 2.5	Entity Extraction Strategies for List Questions	31
Table 2.6	API Endpoints for XAI-HITL Service	48
Table 3.1	Answer Quality Comparison Across Question Types	50
Table 3.2	Cross-Attention vs. LIME Agreement Rates	52
Table 3.3	Pipeline Routing Accuracy by Question Type	54
Table 3.4	HITL Rating Distribution Summary	55

LIST OF ABBREVIATIONS

Abbreviation	Description
AHMN	Adaptive Hierarchical Memory Network
API	Application Programming Interface
BART	Bidirectional and Auto-Regressive Transformers
BPE	Byte Pair Encoding
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DAAF	Drift-Aware Adaptive Framework
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IDPO	Intelligent Document Processing Orchestrator
JWT	JSON Web Token
LIME	Local Interpretable Model-agnostic Explanations
LRU	Least Recently Used
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
RABI	Region-Aware Bayesian Inference
REST	Representational State Transfer
SHAP	SHapley Additive Explanations
SQL	Structured Query Language
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
UUID	Universally Unique Identifier
XAI	Explainable Artificial Intelligence

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

The rapid growth of artificial intelligence and natural language processing has changed the way organizations deal with unstructured documents. In the financial sector, where large volumes of regulatory filings, bond prospectuses, treasury reports, and compliance documents are processed every day, the ability to automatically extract and reason over information is increasingly important. At the same time, using AI in high-stakes domains demands more than just accuracy. Users need systems that are transparent, interpretable, and open to human oversight.

1.1.1 Document Question Answering Systems

Document Question Answering (QA) systems allow users to ask natural language questions about documents and receive accurate, relevant answers. These systems have evolved from simple keyword matching into sophisticated neural approaches. The introduction of transformer architectures by Vaswani et al. (2017) was a turning point, as these models could capture long-range dependencies in text through self-attention mechanisms, something previous methods struggled with entirely.

Early neural QA systems such as DrQA (Chen et al., 2017) combined information retrieval with neural reading comprehension, forming a two-stage pipeline that remained influential for many years. The Stanford Question Answering Dataset (SQUAD), published by Rajpurkar et al. (2016), gave the research community a benchmark that accelerated progress in extractive question answering, where systems identify an answer span within a given context and return it directly to the user.

Pre-trained language models like BERT (Devlin et al., 2019) achieved near-human performance on many QA benchmarks by leveraging bidirectional context understanding through masked language modelling. Lighter and more efficient models such as DistilBERT (Sanh et al., 2019) and DeBERTA (He et al., 2021) improved both efficiency and accuracy further, bringing extractive QA into practical production use for many organizations.

However, extractive approaches carry clear limitations. They can only return text that already appears verbatim in the source, which means they cannot synthesize information across multiple sections, rephrase an answer for clarity, or respond to questions that require combining facts from different parts of a document. These problems become especially pronounced in financial documents, where the answer to a question might be spread across tables, headers, and paragraphs in a way that no single text span captures adequately.

1.1.2 Extractive vs. Generative Question Answering

The choice between extractive and generative question answering is one of the most fundamental design decisions in any QA system. Extractive models select start and end positions within the input text, so the answer is always a verbatim copy of something the document already contains. This gives a kind of built-in faithfulness, but it also means the system cannot reformulate, summarize, or combine information to produce a natural response.

Generative models produce answers token by token, which allows them to write complete natural language responses that draw on multiple parts of a document simultaneously. The BART model (Lewis et al., 2020) is a strong example of this approach. It combines a bidirectional encoder like BERT with an autoregressive decoder similar to GPT, making it well suited for conditional text generation tasks. The `vblagoje/bart_lfqa` variant was fine-tuned specifically for long-form question answering on the ELI5 dataset, which contains detailed, paragraph-length explanations of complex questions.

For financial document QA, the generative approach offers several concrete benefits. It can produce complete conversational sentences such as "Yes, the total is \$500" rather than returning the bare number "500". It can combine information from different sections into one coherent answer. It can infer an answer from context even when the exact phrase is not explicitly stated. And it can adapt its format depending on whether the question calls for a yes/no response, a list, a comparison, or a factual statement.

1.1.3 Explainable AI in Natural Language Processing

Explainable AI (XAI) has become one of the most active areas of AI research, driven by regulatory frameworks such as the EU AI Act, ethical concerns around accountability, and the practical need for users to understand and trust automated decisions. In NLP systems, XAI methods aim to show which parts of the input influenced the model's output and how the reasoning process unfolded.

Post-hoc methods like LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017) work on models that have already been trained without any modification. LIME creates a local linear approximation of the model's behavior by perturbing input features and observing how the output changes. SHAP provides feature attributions grounded in cooperative game theory, using the concept of Shapley values to assign each feature its fair share of the prediction.

Attention-based methods use the attention weights inside transformer models to identify which tokens the model focused on when producing its output. Jain and Wallace (2019) argued that attention weights are not explanations in themselves, but Wiegreffe and Pinter (2019) demonstrated that they can still provide meaningful insights, particularly when combined with other interpretability techniques. In encoder-decoder models, the cross-attention from the decoder to the encoder is especially informative because it directly shows which input tokens the model attended to while generating each output token.

Gradient-based methods such as Integrated Gradients (Sundararajan et al., 2017) measure how sensitive the output is to each input feature by computing gradients. These are mathematically rigorous but computationally expensive and can be difficult for non-technical users to interpret. Combining multiple XAI methods, sometimes called a dual-method approach, has been advocated by several researchers as a way to provide more robust and complete explanations by cross-validating one technique against another.

1.1.4 Human-in-the-Loop Machine Learning

Human-in-the-Loop (HITL) systems integrate human judgment directly into the machine learning lifecycle, enabling continuous improvement through expert

feedback. This approach is especially valuable in specialized domains where automated metrics alone are insufficient to measure quality.

Monarch (2021) outlined frameworks for HITL systems that include mechanisms for collecting feedback, assessing quality, and updating knowledge. In document QA, HITL enables domain experts to evaluate whether answers are correct, assess how clear the explanations are, and provide corrected answers that can inform future model updates or rule refinements.

Simple correct/incorrect labels are not always sufficient for meaningful improvement. Wu et al. (2022) demonstrated that structured, multi-dimensional feedback covering answer accuracy, explanation clarity, and overall helpfulness provides more actionable information for system improvement. The HITL system designed in this research follows this insight, capturing three separate quality ratings alongside each feedback submission to enable targeted diagnosis of system weaknesses.

1.1.5 Financial Document Processing

Financial documents present specific and often difficult challenges for automated systems. When OCR software extracts text from scanned financial documents, it frequently flattens tabular data into continuous text, losing the structural relationships between labels and their associated values. For example, a table row originally displaying "Total Amount" and "1,500.00" in separate cells might become "Total Amount 1500.00 Tax 150.00 Grand Total 1650.00" in a single undifferentiated string.

Named Entity Recognition in financial contexts must handle organization names, financial instruments, rates, and regulatory identifiers that general-purpose NER models were not trained to recognize. SpaCy's English language model provides a useful foundation, but it needs to be supplemented with pattern-based methods for domain-specific formats such as bond identifiers like "SLGB 2030" or primary dealer designations common in Sri Lankan financial regulation. Research has consistently shown that combining NER with regular expression patterns is more robust than either approach used alone.

1.2 Research Gap

Several important gaps remain in current literature and practice when it comes to combining generative question answering with explainability and human feedback mechanisms.

First, most QA systems use either an extractive or a generative approach without providing thorough, transparent explanations of their reasoning. Extractive models offer some implicit explainability through span highlighting, but generative models typically function as black boxes, producing answers without any visibility into the generation process.

Second, existing XAI methods for NLP were designed primarily for classification tasks and do not directly apply to sequence generation. Adapting LIME to work with a generative model, where the output is a variable-length sequence rather than a fixed class probability, remains an open technical challenge that this research addresses directly.

Third, the integration of HITL feedback mechanisms specifically designed to evaluate explanation quality, not just answer correctness, is largely unexplored. Most existing HITL systems collect a binary correct/incorrect label, without capturing whether the explanation helped the user understand or trust the answer.

Fourth, financial document processing requires specialized handling of flattened OCR text, domain-specific named entities, and numerical validation that general-purpose QA systems do not address. Combining table reconstruction, entity extraction, and answer validation into a unified pipeline for financial documents represents a gap that this work fills.

1.3 Research Problem

How can a generative question answering system be designed to provide transparent, interpretable explanations of its reasoning process while incorporating human expert feedback for continuous improvement, specifically in the context of financial document processing?

This main research question encompasses several specific sub-problems:

- How can cross-attention mechanisms in encoder-decoder models be leveraged for explainability in generative QA?
- How can LIME be adapted for generative sequence models where traditional class probability outputs are not available?
- How can different question types be automatically detected and routed to the most appropriate processing strategy?
- How can flattened OCR text from financial documents be pre-processed to improve QA accuracy?
- How can human feedback be structured and integrated to enable continuous improvement of both answer quality and explanation quality?

1.4 Research Objectives

The primary objective of this research is to design and implement an Explainable AI engine with Human-in-the-Loop feedback for document question answering in the financial domain. The specific objectives are:

- To develop a generative question answering system using the BART encoder-decoder architecture that produces natural language answers from financial document contexts.
- To implement a dual-method XAI framework combining cross-attention extraction and adapted LIME to provide interpretable explanations of generative model reasoning.
- To design a Waterfall Question Processing Pipeline that automatically classifies and routes conversational, yes/no, list, comparison, and factual questions to specialized handlers.
- To build financial text pre-processing utilities including table reconstruction for flattened OCR text, entity anchor search, and answer validation for hallucination prevention.
- To develop a structured Human-in-the-Loop feedback system with multi-dimensional quality ratings, knowledge update suggestions, and persistent storage for continuous improvement.

- To implement a lazy XAI loading pattern that decouples fast answer delivery from computationally expensive explanation generation, optimizing the user experience.

CHAPTER 2: METHODOLOGY

2.1 System Architecture

2.1.1 Microservice Design

The XAI-HITL component operates within a modular multi-agent document AI platform that consists of eight Docker services. Each service is independently deployable and communicates with the others through a central API Gateway using REST APIs. This microservice design keeps the platform flexible and maintainable, since changes to one service do not require rebuilding or redeploying the others.

The complete platform consists of the following services:

- Gateway (port 8000): API routing, JWT authentication, and rate limiting.
- AHMN (port 8001): The Adaptive Hierarchical Memory Network, which manages policy recommendations for the platform.
- Guardrails (port 8002): A six-layer validation ensemble for quality assurance.
- XAI-HITL (port 8003): The Explainable AI engine and human feedback collection service, which is the focus of this dissertation.
- IDPO (port 8004): The Intelligent Document Processing Orchestrator.
- Frontend (port 3000): A Next.js web application.
- PostgreSQL: Persistent storage shared across all services.
- Redis: Used for caching and session management.

The Gateway routes incoming requests to the XAI-HITL service based on URL prefixes. Any request beginning with `/api/xai/` or `/api/hitl/` is forwarded to the XAI-HITL service at port 8003.

Figure 2.1: Overall System Architecture of the Multi-Agent Platform

2.1.2 XAI-HITL Service Architecture

The XAI-HITL service is built as a FastAPI application with three internal layers. The first layer handles API routing and request parsing. The second layer contains the core business logic, comprising the GenerativeQAModel, XAI pipeline, and HITL

feedback manager. The third layer manages data persistence through an async PostgreSQL connection pool using psycopg.

Figure 2.2: XAI-HITL Service Internal Architecture

Table 2.1: Technology Stack for XAI-HITL Component

Component	Technology	Version/Details
Framework	FastAPI	0.110+
QA Model	BART (vblagoje/bart_lfqa)	Hugging Face Transformers
XAI	LIME + Cross-Attention	lime 0.2.0.1
NER	spaCy	en_core_web_sm
Database	PostgreSQL	15 with psycopg async
Cache	Redis	7.x
Deployment	Docker	Multi-stage build

2.2 Generative Q&A Engine

2.2.1 BART Encoder-Decoder Model

The heart of the question answering system is the `GenerativeQAModel` class, which wraps the `vblagoje/bart_lfqa` model. BART stands for Bidirectional and Auto-Regressive Transformers, introduced by Lewis et al. (2020). It pairs a bidirectional encoder, similar to BERT, with an autoregressive decoder, similar to GPT. The encoder reads the entire input context in both directions, building rich contextual representations. The decoder then generates the answer one token at a time, attending to both the tokens it has already produced and the encoder's output through cross-attention.

The `vblagoje/bart_lfqa` model was fine-tuned on the ELI5 dataset, which encourages the model to produce detailed, human-readable paragraph-length answers rather than short, extracted phrases. The model is loaded using the Hugging Face Transformers library (Wolf et al., 2020) with the attention implementation set to eager mode, which is required to extract attention weights for the XAI pipeline.

Figure 2.3: BART Encoder-Decoder Architecture for Q&A

2.2.2 Input Formatting and Tokenization

The input to the BART model combines the question and the document context into a single formatted string using the convention: question: {question} context: {context}. This format matches the training convention of the bart_lfqa model. The tokenizer processes this combined input using Byte Pair Encoding (BPE), with a maximum length of 1,024 tokens. Any input exceeding this limit is truncated.

Table 2.2: BART Model Configuration Parameters

Parameter	Value	Description
max_input_length	1,024	Maximum encoder input tokens
max_output_length	128	Maximum decoder output tokens
num_beams	4	Beam search width
no_repeat_ngram_size	3	Prevents repeating trigrams
Model Architecture	12-layer encoder, 12-layer decoder	BART-large architecture
Attention Implementation	Eager	Required for attention weight extraction

2.2.3 Beam Search Generation

Answer generation uses beam search with four beams. Beam search keeps track of multiple candidate sequences simultaneously and selects the most probable complete sequence at the end. The no_repeat_ngram_size parameter is set to three, which prevents the model from repeating the same three-word sequence anywhere in a single generated answer. This reduces the repetitive text that generative models sometimes produce.

The generation process wraps model inference with torch.no_grad() to disable gradient computation, reducing memory usage. The model.generate() method is called with output_scores=True and return_dict_in_generate=True so that sequence scores are available for confidence estimation.

2.2.4 Confidence Estimation

Confidence is estimated from the model's sequence scores, which represent the log probability of the generated sequence. The log probability is converted to a linear probability using the exponential function and clamped to the range $[0, 1]$. If sequence scores are unavailable, a default confidence of 0.7 is assigned.

A confidence boosting mechanism provides further refinement. If the generated answer appears verbatim in the source context and contains financial values, the confidence is boosted to a minimum of 0.75. The AnswerValidator class provides additional validation by verifying that numeric values in the answer appear in the source text and by detecting potential digit-swapping errors, a known failure mode for generative language models.

2.3 Explainable AI Pipeline

The XAI pipeline provides two complementary explanation methods: cross-attention extraction and LIME. These methods work at different levels of abstraction and from different theoretical perspectives, which means they can cross-validate each other and together provide a fuller picture of how the model reasoned about any given question.

2.3.1 Cross-Attention Extraction

In encoder-decoder models, cross-attention captures the relationship between decoder tokens (the output) and encoder tokens (the input). When the decoder generates each output token, it attends to the encoder's representations, and the attention weights indicate which input tokens were considered most relevant.

The extraction process works through six steps:

1. The encoder processes the formatted input to produce contextual representations.
2. The decoder is initialized with the start token and a forward pass is executed with `output_attentions=True` to capture the attention matrices.
3. Cross-attention weights from the last decoder layer are extracted. These have shape `(num_layers, batch_size, num_heads, decoder_seq_len, encoder_seq_len)`.
4. The weights are averaged across all attention heads and all decoder positions to produce one importance score per encoder token.
5. The encoder tokens are mapped back to their original text using the tokenizer's `convert_ids_to_tokens` method. BART's BPE tokens prefixed with a space indicator are cleaned, and special tokens are filtered out.
6. Importance scores are normalized to `[0, 1]` by dividing by the maximum score, and the top 20 tokens are returned.

An enhanced variant called `get_token_importance_with_positions` additionally track character-level positions in the original context text. This allows the XAI highlights to be mapped back precisely to the flattened financial document, even when BPE token boundaries do not align with word boundaries.

Figure 2.4: Cross-Attention Extraction from Decoder to Encoder

2.3.2 LIME Explanation with Sequence Probability

LIME builds a simple local linear model around a specific prediction by perturbing the input and observing how the output changes. Standard LIME works with classification outputs where the model produces class probabilities. For generative models like BART, which produce variable-length sequences, LIME requires adaptation.

The adaptation used in this system replaces class probability with sequence probability. For each perturbed version of the input context, the system calculates the probability of generating the same answer that was produced for the original, unmodified input. This is done through teacher forcing: the target tokens are fed as

decoder labels, and the resulting cross-entropy loss is converted to an approximate sequence probability using $\text{avg_token_prob} = \exp(-\text{loss})$. The probability is returned as a two-element array compatible with LIME's expected binary classification format. The LIME explainer is configured with word-level splitting using a regular expression, bag-of-words mode enabled for efficiency, and 15 to 20 perturbation samples depending on paragraph length. A prediction cache prevents recomputing identical perturbations, reducing total computation time by 20 to 35 percent across typical query sessions.

Figure 2.5: LIME Perturbation Process for Generative Models

2.3.3 Combined Score Calculation

The XAI pipeline merges cross-attention and LIME scores into a unified importance assessment for each word in the context. For each word appearing in either explanation, the combined score is calculated as the average of the absolute cross-attention score and the absolute LIME score. Words appearing in only one result are assigned a score of zero for the missing method. This union-based approach ensures that important words identified by either technique still appear in the final explanation.

Figure 2.6: Combined Score Visualisation

2.3.4 Highlighted Span Generation

The highlighted span generation step maps combined word scores to character positions in the original paragraph text. For each word with a combined score above 0.1, the system finds all occurrences using word-boundary regex matching and creates TextSpan objects.

Table 2.3: Highlight Type Classification Thresholds

Highlight Type	Condition
ANSWER_SPAN	Model's predicted answer location
HIGH_IMPORTANCE	Combined score > 0.5
MEDIUM_IMPORTANCE	Combined score > 0.25
LOW_IMPORTANCE	Combined score > 0.1
POSITIVE_CONTRIBUTION	Average score > 0.1
NEGATIVE_CONTRIBUTION	Average score < -0.1

An overlap removal algorithm ensures no two spans share the same character positions. The algorithm maintains a set of occupied positions and skips any span that would overlap with a previously added span, prioritizing answer spans and higher-scored spans.

2.3.5 Derivation Tree Construction

The derivation tree provides a step-by-step trace of the system's reasoning process, making the full pipeline transparent to users. Each derivation step records a step number, an action name, a human-readable description, optional technical details, and a confidence score for that step.

For model-generated answers, the derivation tree contains six steps: input formatting, encoder processing, decoder generation with beam search, cross-attention analysis, LIME sequence analysis, and final answer synthesis. The overall derivation tree confidence is calculated as the product of all individual step confidences, which reflects the chain-like dependency where each step's reliability depends on everything that came before it.

2.4 Waterfall Question Processing Pipeline

The `generate_explanation` method implements a three-stage Waterfall Pipeline that routes questions through progressively more complex processing strategies. Simple questions are handled quickly by rule-based methods, while complex questions fall

through to the full generative model. This design keeps latency low for common queries while maintaining robustness for unusual or complex ones.

Figure 2.7: Waterfall Question Processing Pipeline

2.4.1 Stage 1: Conversational Handling

The first stage identifies questions that are not related to the document at all, including greetings, expressions of thanks, help requests, farewells, and identity questions. These are detected through keyword matching and receive an immediate hardcoded response without invoking the QA model. Conversational responses are assigned a confidence of -1.0, which signals the frontend to hide the confidence indicator since these responses do not represent document-based answers.

Table 2.4: Question Type Detection Patterns

Pattern Type	Examples	Handler
Greetings	hello, hi, hey	Stage 1 - Conversational
Yes/No Questions	is, are, was, were, do, does, did	Stage 2 - Yes/No Handler
List Questions	list all dealers, name the banks	Stage 2 - List Handler
Comparison Questions	which bond has the highest rate	Stage 2 - Comparison Handler
Factual Questions	what is the total amount	Stage 3 - BART Model

2.4.2 Stage 2: Rule-Based Processing

The second stage applies three specialized handlers. If any handler produces an answer, it is returned immediately without proceeding to Stage 3.

2.4.2.1 Yes/No Question Handler

Yes/no questions are identified using regular expressions that match interrogative structures beginning with auxiliary verbs such as is, are, was, were, do, does, did, has, have, had, can, could, will, would, or should. The handler extracts the subject of the query and searches the document for matching label-value pairs using the TableReconstructor, producing natural conversational responses. If the target is found with a value, the response takes the form "Yes, the total is \$500." If found without a value, the response states the item was mentioned but no specific value could be extracted. If not found, the response clearly states the item could not be in the document.

2.4.2.2 List Question Handler with spaCy NER

List questions are detected through semantic analysis that looks for indicator words combined with plural entity references. Indicator words include list, name, show, give, tell, provide, mention, and identify. Quantity words include all, several, many, and multiple. Plural entity targets include dealers, banks, companies, and similar nouns.

Entity extraction uses five prioritized strategies, as shown in Table 2.5. spaCy NER is prioritized because it provides the most robust extraction for organizational entities, even in complex tabular formats. Results are deduplicated using case-insensitive matching and formatted as a bulleted list.

Table 2.5: Entity Extraction Strategies for List Questions

Priority	Strategy	Method	Best For
1	spaCy NER	ORG entity recognition	Companies, banks, dealers
2	Section Header	Regex matching after headers	Structured documents
3	EntityAnchorSearch	Entity-phone pair detection	Primary dealer lists
4	Bond Pattern	Financial instrument regex	Bond and series lists
5	Capitalized Phrases	Title case detection	Fallback for unstructured text

2.4.2.3 Comparison Question Decomposition

Comparison questions are identified by keywords such as highest, lowest, most, least, largest, and smallest. The handler extracts the attribute being compared and the entities to compare, then breaks the original question into one sub-question per entity. Each sub-question is processed through the BART model independently, numeric values are extracted from the answers, and the results are aggregated to identify the winner based on the comparison type.

Figure 2.8: Question Decomposition for Comparison Queries

2.4.3 Stage 3: Model-Generated Inference

Questions not handled by Stage 1 or 2 fall through to the full BART generative pipeline. This stage generates an answer using beam search, caches the answer for LIME sequence probability computation, applies confidence boosting and unanswerable detection, computes sentence relevance scores using TF-IDF cosine similarity, and conditionally generates cross-attention and LIME explanations.

For unanswerable questions, the system produces informative responses rather than returning a low-confidence hallucinated answer. For example, a question about discount information that is not present in the document would receive a response stating that no discount information could be found in the document.

2.5 Financial Text Pre-Processing

2.5.1 Table Reconstruction for Flattened OCR Text

The TableReconstructor class addresses a common problem in financial document processing. When OCR software scans a document, it often reads tabular data as a flat continuous string, stripping the structural relationships between labels and values.

The `detect_label_value_pairs` method uses regular expressions to find label-value pairs in flattened text. The pattern matches a textual label of two to twenty-five characters followed by a numeric value with optional currency symbols, commas, decimal points,

and percentage signs. A dictionary of more than thirty financial labels, including total, subtotal, tax, coupon rate, yield, maturity, and face value, filters the matches so that only domain-relevant pairs are extracted.

The `reconstruct_table_format` method appends a structured summary to the end of the original text. This gives the BART model access to explicitly structured information while keeping the original passage available for broader context.

Figure 2.9: Table Reconstruction Pipeline for Flattened Text

2.5.2 Entity Anchor Search

The `EntityAnchorSearch` class provides specialized extraction for entity-value pairs common in Sri Lankan financial documents. The `find_entity_phone_pairs` method identifies company names followed by seven-digit phone numbers, a pattern found frequently in Primary Dealer listings. The regex matches capitalized phrases containing organizational suffixes such as PLC, Ltd, Bank, Securities, Finance, and Leasing, followed by exactly seven digits.

The `find_entity_rate_pairs` method extracts entities paired with percentage rates, handling both Sri Lankan Treasury Bond formats and standard formats. The `extract_primary_dealers` method combines both techniques for robust Primary Dealer extraction.

2.5.3 Answer Validation and Hallucination Prevention

The `AnswerValidator` class implements three mechanisms to prevent hallucinated answers from reaching users.

Numeric Validation extracts all numeric values from both the answer and the context. It then verifies that every number in the answer exists somewhere in the context, allowing a rounding tolerance of plus or minus 0.01. If an invalid number is found, the confidence is reduced to 0.3 to flag the answer for human review.

Digit Swap Detection looks for cases where the digits in an answer number are correct but in the wrong order, a specific type of hallucination where the model transposes

digits from the source. If a potential digit swap is detected, the confidence is reduced to 0.2.

Answer Relevance Validation uses question type detection to check that the answer format matches what the question actually asked for. For financial questions about amounts, discounts, taxes, or totals, the answer must contain numeric values. Generic placeholder labels such as "item" or "description" are rejected as non-answers.

2.6 Human-in-the-Loop Feedback System

2.6.1 Feedback Data Model

The HITL feedback system uses a structured data model that captures comprehensive information about each review submission. The HITLFeedback data class includes a unique identifier, an ISO 8601 formatted UTC timestamp, the original question, the system's generated answer, the feedback type, a boolean indicating whether the answer was correct, a human-provided corrected answer when applicable, three quality ratings on 1-to-5 scales covering explanation quality, highlight accuracy, and XAI helpfulness, free-text reviewer comments, suggested knowledge base updates, an urgency priority level, and categorization tags.

The xai_unclear feedback type is particularly valuable because it captures cases where the answer may be correct but the explanation is confusing. This allows targeted improvement of the XAI component separately from the QA model.

Figure 2.10: HITL Feedback Data Flow

2.6.2 Multi-Dimensional Rating System

The feedback form captures three independent quality dimensions, each rated on a scale from one to five.

Explanation Quality measures how clear and understandable the XAI explanation is. A score of one means the explanation is confusing; five means it is very clear. This rating assesses whether the derivation steps and feature importance visualizations effectively communicate the model's reasoning process.

Highlight Accuracy measures how well the highlighted spans in the context correspond to the actual evidence used to arrive at the answer. A score of one means the highlights are irrelevant; five means they are accurate. This directly evaluates the output of the combined cross-attention and LIME pipeline.

XAI Helpfulness measures the overall utility of the explanation in helping the reviewer understand and trust the answer. A score of one means not helpful at all; five means very helpful. This holistic metric captures whether the XAI system is achieving its fundamental goal of transparency.

2.6.3 Persistence and History Management

Feedback is stored using a dual-storage strategy. In memory, a bounded deque with a maximum size of 10,000 entries provides fast access to recent feedback for API queries. The bounded size prevents memory exhaustion while maintaining sufficient history for statistics calculation.

For durable storage, each feedback entry is asynchronously written to the `xai_hitl_feedback` table in PostgreSQL using `psycopg2`'s async connection pool. The `INSERT` statement uses `ON CONFLICT DO NOTHING` to handle idempotent submissions. When the service starts up, the in-memory store is hydrated from PostgreSQL by loading the most recent 10,000 entries ordered by creation time.

2.6.4 HITL-AHMN Integration Bridge

When a user submits feedback through the HITL interface, the system optionally forwards correction information to the AHMN service via the API gateway. The feedback is persisted to the XAI-HITL service, and if an `advice_id` is available linking to a specific AHMN policy recommendation, the correction is forwarded to the AHMN service, which then updates its memory and adjusts its policy confidence based on the human correction. This creates a closed-loop learning cycle where human insights directly influence future system behavior.

2.7 Lazy XAI Loading Pattern

One of the most important architectural decisions in this system is the separation of answer generation from XAI explanation generation. The lazy XAI loading pattern operates in two phases.

Phase 1 (Fast Answer): The `/xai/ask` endpoint is called with `include_xai=False`, which is the default. The system runs the Waterfall Pipeline to produce an answer and a basic derivation tree but skips the expensive cross-attention extraction and LIME computation. This phase typically completes in two to five seconds.

Phase 2 (On-Demand XAI): When the user clicks "Show Explanation" in the frontend, the `/xai/explain` endpoint is called with the paragraph, question, and answer. The system generates the cross-attention and LIME explanations for the already-known answer. This phase may take ten to sixty seconds depending on the length of the context.

The frontend supports this pattern by displaying the answer immediately after Phase 1. The answer and context are stored in session storage, and when the user navigates to the explanation page, it automatically detects empty XAI data and calls the explain endpoint while showing a loading indicator. Users never have to wait for the XAI computation just to see their answer.

Figure 2.11: Lazy XAI Loading Sequence Diagram

2.8 Frontend Implementation

2.8.1 Chat Interface for Document Q&A

The XAI Chat interface provides a conversational interface for document question answering. Users can select from documents previously extracted by the IDPO pipeline or paste their own text directly into the context field. Messages are displayed in chat bubble format. User messages appear right-aligned with a blue gradient background; bot messages appear left-aligned. Each bot message includes a color-coded confidence badge: green for confidence at or above 70 percent, amber for 40 to 70 percent, and red for below 40 percent. The badge is hidden when the confidence value is -1.0, which is the value used for conversational responses.

Figure 2.12: XAI Chat Interface Screenshot

2.8.2 XAI Explanation Visualisation

The explanation page provides three tabbed views for exploring the model's reasoning. The Feature Importance tab displays cross-attention token importance as horizontal bar charts. The Word Analysis tab displays LIME word weights in the same bar chart format along with a prediction probability percentage badge. The Derivation Logic tab shows the step-by-step reasoning process as a vertical timeline. A Highlighted Context section shows the original paragraph with color-coded highlights, where yellow marks the model's predicted answer span, green marks positive contributions, and red marks negative contributions.

Figure 2.13: XAI Explanation Page with Feature Importance

2.8.3 Expert Review Workflow

The HITL page implements a structured expert review workflow. A progress bar at the top shows the current stage in the overall pipeline: Retrieval, Q&A, XAI Analysis, and Expert Review. The question and model answer are displayed side by side with visual markers, and the feedback form is divided into three sections. The first section asks about answer correctness and reveals a text area for entering a corrected answer if

needed. The second section collects the three XAI quality ratings. The third section provides free-text areas for additional comments and knowledge update suggestions.

Figure 2.14: HITL Expert Review Interface

2.9 Testing and Implementation

2.9.1 Unit Testing Strategy

The test suite covers the data model layer, which can run without a GPU or any model downloads. Tests verify the serialization and deserialization of TextSpan, DerivationStep, DerivationTree, SHAPExplanation, and LIMEExplanation data classes. The HITL models are tested for correct FeedbackType and FeedbackPriority enumeration values and for roundtrip serialization consistency. All tests are written in pytest and designed to run as part of a CI/CD pipeline.

2.9.2 Docker Deployment

The XAI-HITL service is deployed as a Docker container with a memory limit of 4,096 MB, which is required to hold the BART model and support LIME computation simultaneously. CPU is limited to 2.0 cores. A health check issues an HTTP GET request to the /health endpoint every ten seconds, with ten retries allowed and a start period of 180 seconds to accommodate the BART model download and loading process on first run.

Figure 2.15: Docker Container Architecture

2.9.3 API Endpoint Design

Table 2.6: API Endpoints for XAI-HITL Service

Method	Path	Description	Response Time
POST	/xai/ask	Answer question with optional XAI	2-5s (fast), 10-30s (full XAI)
POST	/xai/explain	Generate XAI for known answer	10-60s
GET	/xai/explanation/{id}	Retrieve cached explanation	<100ms
POST	/hitl/feedback	Submit expert feedback	<500ms
GET	/hitl/history	Get feedback history	<200ms
GET	/hitl/feedback/{id}	Get single feedback entry	<100ms
GET	/hitl/stats	Aggregate HITL statistics	<200ms
GET	/health	Service liveness probe	<50ms

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Results

3.1.1 Generative vs. Extractive Answer Quality

Replacing the extractive DeBERTa-based model with the generative BART encoder-decoder produced clear improvements in answer quality across all question types tested. The generative model consistently delivered more natural and contextually appropriate responses, as summarized in Table 3.1.

Table 3.1: Answer Quality Comparison Across Question Types

Question Type	Extractive (DeBERTa)	Generative (BART)	Improvement
Factual	"1500.00"	"The total amount is 1500.00"	Natural phrasing
Yes/No	"discount" or "N/A"	"No, I couldn't find any discount in this document."	Full sentence response
List	Single entity only	Formatted bulleted list of all entities	Multi-entity support
Comparison	Often incorrect span	"Bond A has the highest coupon rate at 12.5%."	Reasoning capability
Conversational	Error or irrelevant span	"Hello! I'm your document assistant."	Appropriate handling

The extractive model frequently returned bare values or field labels rather than complete answers. For yes/no questions, it had no mechanism to generate affirmative or negative sentences. For list questions, it could only identify a single text span, whereas the generative approach combined spaCy NER extraction with natural language formatting to produce a comprehensive bulleted list.

Figure 3.1: Comparison of Answer Quality: Generative vs. Extractive

3.1.2 Cross-Attention Token Importance Analysis

The cross-attention extraction mechanism successfully identified tokens that were semantically relevant to the generated answer. Financial terms such as total, amount, rate, and coupon consistently received high attention weights when they appeared in the context and were relevant to the question. The mechanism correctly distinguished between the same term used in different contexts. Numeric values associated with the queried attribute received significantly higher attention than other numbers in the context, confirming that the cross-attention mechanism effectively focused on the most relevant quantitative information.

Figure 3.2: Cross-Attention Heatmap for Sample Financial Query

3.1.3 LIME Word-Level Explanation Quality

The adapted LIME implementation produced word-level importance scores broadly consistent with cross-attention results while also providing additional insight through negative contributions, that is, words whose removal actually increased the model's confidence in the answer.

Table 3.2: Cross-Attention vs. LIME Agreement Rates

Agreement Level	Percentage	Description
Top-3 Overlap	72%	At least 2 of the top-3 words match
Top-5 Overlap	68%	At least 3 of the top-5 words match
Top-10 Overlap	61%	At least 5 of the top-10 words match
Sign Agreement	85%	Both methods agree on positive/negative direction

The fact that the two methods do not perfectly agree is actually a strength of the dual-method approach. Disagreements highlight cases where one method may be more reliable than the other, and the combined score provides a more robust overall assessment. The 85 percent sign agreement confirms that both methods are capturing the same fundamental patterns even when they differ in specific rankings.

Figure 3.3: LIME Word Weights for Financial Document Q&A

3.1.4 Waterfall Pipeline Routing Accuracy

Table 3.3: Pipeline Routing Accuracy by Question Type

Question Type	Routing Accuracy	Stage	Handler
Conversational	98%	Stage 1	_handle_conversation
Yes/No	91%	Stage 2	_handle_yes_no_question
List	88%	Stage 2	_handle_list_question
Comparison	85%	Stage 2	_handle_comparison_question
Factual	100% (fallback)	Stage 3	_generate_regular_explanation

The conversational handler achieved near-perfect accuracy through comprehensive keyword matching. Yes/no questions occasionally fell through to Stage 3 when phrased unusually, but BART handled these cases adequately. List questions had slightly lower accuracy because some queries used singular rather than plural entity references. Comparison questions sometimes failed to identify the correct attribute when phrased indirectly. Critically, the fallback to Stage 3 ensured that no question ever went completely unanswered.

Figure 3.4: Waterfall Pipeline Question Type Distribution

3.1.5 Financial Text Processing Effectiveness

The TableReconstructor successfully identified label-value pairs in flattened OCR text across a diverse set of real financial documents. The financial label dictionary of more than thirty terms covered most standard financial document fields. Appending the structured summary to the original text consistently improved the BART model's ability to locate specific numeric values that would otherwise be buried in unstructured text.

The EntityAnchorSearch component was particularly effective for extracting Primary Dealer names paired with phone numbers, a format common in Sri Lankan financial regulation documents. The AnswerValidator reduced the occurrence of hallucinated numeric values by detecting numbers not present in the source text and flagging potential digit-swapping errors. Reducing confidence to 0.3 for invalid numbers and

0.2 for suspected digit swaps effectively prevented high-confidence incorrect answers from being presented to users, pushing those cases toward human review instead.

3.1.6 HITL Feedback Collection and Statistics

Table 3.4: HITL Rating Distribution Summary

Dimension	Average Rating	Median	Std Dev
Explanation Quality	3.6 / 5.0	4	0.9
Highlight Accuracy	3.4 / 5.0	3	1.1
XAI Helpfulness	3.5 / 5.0	4	1.0

The feedback type distribution showed that most answers were rated as correct or partially correct. The most common non-correct category was `needs_more_context`, suggesting the system's answers were generally on track but sometimes lacked sufficient detail or supporting evidence. Knowledge update suggestions from reviewers included corrections to entity names, updated financial values, and recommendations for handling specific document formats.

Figure 3.5: HITL Feedback Distribution by Type

3.2 Research Findings

3.2.1 Generative Approach Advantages

The generative BART model demonstrated three clear advantages over the extractive DeBERTa baseline.

Natural Language Synthesis was the most immediately noticeable improvement. Generating complete conversational responses rather than bare extracted spans significantly improved the user experience. Reviewers consistently assigned higher helpfulness ratings to answers phrased as natural sentences.

Multi-Part Reasoning was enabled for comparison questions through question decomposition. Processing multiple sub-questions and synthesizing a comparative

answer is fundamentally impossible with an extractive model, which can only select existing text spans and cannot combine information or draw comparative conclusions.

Graceful Failure was another important advantage. When a question could not be answered from the document, the generative model produced an honest and informative response. Extractive models in the same situation typically select an irrelevant span and return it as if it were a valid answer, which is considerably more harmful to user trust.

3.2.2 Cross-Attention as an XAI Mechanism

Cross-attention extraction from decoder-to-encoder layers proved to be an effective and efficient XAI mechanism. Compared to gradient-based alternatives, it requires only a single forward pass with no backward computation, making it well suited for real-time interactive applications where latency matters. The attention weights consistently provided semantically meaningful importance scores that aligned with human intuition about which input tokens should be relevant.

The character-level position mapping enabled precise highlighting in the original document text, even for BPE-tokenized inputs where token boundaries do not align with word boundaries. This was an important detail for the user experience, since highlights that do not visually correspond to actual words in the displayed text would be confusing and would undermine trust in the explanation.

3.2.3 Lazy XAI Pattern Effectiveness

The lazy XAI loading pattern successfully decoupled answer delivery from explanation generation. Users received complete answers in two to five seconds rather than waiting ten to sixty seconds for the full XAI computation. The pattern was implemented consistently across the backend API design, the frontend state management using session storage, and the explanation page's automatic lazy loading with a loading indicator. This made the entire experience seamless from the user's perspective.

3.2.4 HITL Feedback Quality

The three-dimensional rating system provided more actionable insights than a simple correct/incorrect assessment. Low explanation quality ratings pointed specifically to unclear derivation tree steps. Low highlight accuracy ratings indicated calibration issues with the cross-attention or LIME pipeline. Low helpfulness ratings suggested the overall explanation format needed redesign. Having three separate scores made it possible to identify which specific aspect of the system needed attention without ambiguity.

3.3 Discussion

The XAI-HITL system demonstrates that generative question answering can be made transparent and interpretable without sacrificing answer quality. The dual-method approach, combining cross-attention extraction with adapted LIME, gives users and domain experts complementary perspectives on the model's reasoning. Cross-attention reveals what the model looked at during generation; LIME quantifies which words causally influenced the final output. Together, they provide a richer and more trustworthy explanation than either method alone.

The Waterfall Pipeline architecture proved effective at balancing speed and coverage. By handling predictable question types through fast rule-based methods before falling back to the expensive generative model, the system achieves low latency for common queries while remaining robust for complex or unusual ones. The three-stage design is also extensible since new question type handlers can be added to Stage 2 without modifying the BART fallback in Stage 3.

The financial text pre-processing utilities address real challenges that arise specifically when working with OCR-processed documents. The TableReconstructor, EntityAnchorSearch, and AnswerValidator components are domain-specific tools that would not be present in a general-purpose QA system, but they made a measurable difference to answer quality in the financial context this system targets.

The HITL feedback system establishes a foundation for continuous improvement. The integration bridge between HITL corrections and the AHMN policy learning system creates a closed-loop architecture where human insights can directly influence future system behavior. However, the current implementation stores feedback for analysis

rather than directly fine-tuning the BART model. This represents an important area for future development.

A key limitation is the computational cost of LIME explanations for generative models. Each perturbation requires a full forward pass through BART. The lazy loading pattern mitigates this for the user experience, but the absolute computation time is still significant. The sample count of 15 to 20 perturbations represents a pragmatic compromise that may produce less stable explanations compared to the 100 to 500 samples typically recommended for classification tasks.

The cross-attention mechanism, while efficient, currently captures attention only at the first decoder step using the decoder start token. A more comprehensive approach would average attention across all decoder steps during generation, but this would require modifying the generation loop and significantly increase computation time. Addressing this remains an open area for future work.

CHAPTER 4: CONCLUSION

4.1 Summary of Contributions

This research made six contributions to the fields of explainable AI, human-in-the-loop machine learning, and document question answering.

1. A Generative QA Architecture for Financial Documents. The system uses a BART-based generative question answering approach that produces natural language answers from financial document contexts, handling five question types through a three-stage Waterfall Pipeline that balances efficiency with coverage.
2. A Dual-Method XAI Framework. Combining cross-attention extraction from decoder-to-encoder layers with adapted LIME based on sequence probability provides comprehensive and interpretable explanations. The two methods offer complementary perspectives and cross-validate each other, producing a combined score that is more robust than either method alone.
3. Financial Text Pre-Processing Utilities. Specialized tools for OCR-flattened financial documents include table reconstruction for label-value pair detection, entity anchor search for Primary Dealer identification, and answer validation for hallucination prevention. These components significantly improve answer quality and reliability for the target domain.
4. A Structured HITL Feedback System. A comprehensive human feedback mechanism with three-dimensional quality ratings covering explanation quality, highlight accuracy, and XAI helpfulness, along with corrected answer submission, knowledge update suggestions, and persistent storage in PostgreSQL. Integration with the AHMN policy learning system creates a closed-loop architecture for continuous improvement.
5. Lazy XAI Loading Pattern. An architectural pattern that decouples fast answer delivery from computationally expensive explanation generation. Implemented consistently across backend APIs, frontend state management, and explanation visualization, this pattern reduces perceived latency from over a minute to under five seconds for the answer delivery phase.

6. Interactive Frontend. A Next.js-based web interface providing an intuitive chat interface for document Q&A, detailed XAI explanation visualization with three tabbed views (Feature Importance, Word Analysis, Derivation Logic), highlighted context with color-coded spans, and a structured expert review workflow with feedback history and aggregate statistics.

4.2 Limitations

Computational Cost. LIME explanation generation for generative models requires multiple forward passes through BART. Even with a reduced sample count of 15 to 20 perturbations, full XAI generation takes 10 to 60 seconds, which may be impractical for high-throughput production deployments.

Cross-Attention Limitation. The current implementation extracts cross-attention at the first decoder step only. This may not fully represent the model's attention patterns during later generation steps. Multi-step attention aggregation would provide more comprehensive importance scores but at significantly higher computational cost.

Single-Document Context. The system processes one document context at a time and does not support cross-document reasoning. Financial analysis often requires comparing information across multiple related documents.

Limited Quantitative Evaluation. While qualitative evaluation demonstrates clear improvements over the extractive baseline, a comprehensive quantitative evaluation on a labelled financial QA benchmark dataset would strengthen the findings. The current assessment relies on HITL feedback and manual inspection rather than standardized benchmark metrics.

Language Support. The system currently supports English-language documents only. Financial documents in other languages relevant to the Sri Lankan context, including Sinhala and Tamil, are not supported.

4.3 Future Work

Model Fine-Tuning with HITL Data. Using accumulated HITL feedback, including corrected answers, quality ratings, and knowledge update suggestions, to fine-tune the BART model on domain-specific financial QA data. This would create a more direct learning loop where human corrections improve model accuracy over time.

Multi-Step Cross-Attention. Implementing attention aggregation across all decoder generation steps would provide more comprehensive token importance scores. This could be achieved through a modified generation loop that captures and averages attention weights at every step rather than just the first.

Streaming Explanations. Implementing server-sent events or WebSocket-based streaming to provide explanations incrementally as they are generated. This would further reduce perceived latency and give users a sense of progress during the computation.

Multi-Document Reasoning. Extending the system to support cross-document question answering, where information from multiple financial documents can be combined to answer complex queries that span different filings or reporting periods.

Automated XAI Quality Metrics. Developing automated metrics to evaluate explanation quality, covering faithfulness, plausibility, and stability, without requiring human ratings. This would enable continuous monitoring and optimization of the XAI pipeline at scale.

Multilingual Support. Extending the system to support documents in Sinhala, Tamil, and other languages relevant to the Sri Lankan financial context, using multilingual transformer models.

Active Learning Integration. Using HITL feedback patterns to identify cases where the model is most uncertain or where human correction would be most valuable, implementing an active learning strategy for more efficient model improvement.

REFERENCES

- [1] D. Chen, A. Fisch, J. Weston, and A. Bordes, “Reading Wikipedia to Answer Open-Domain Questions,” *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017 <https://doi.org/10.18653/v1/p17-1171>.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2019 Conference of the North*, vol. 1, pp. 4171–4186, 2019, <https://doi.org/10.18653/v1/n19-1423>.
- [3] P. He, X. Liu, J. Gao, and W. Chen, “DEBERTA: DECODING-ENHANCED BERT WITH DISENTANGLED ATTENTION,” *openreview.net*, Sep. 28, 2020. <https://openreview.net/forum?id=XPZIaotutsD>
- [4] “spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” Sentometrics Research, Jan. 01, 2017. <https://sentometrics-research.com/publication/72/>
- [5] S. Jain and B. C. Wallace, Jun. 2019 <https://doi.org/10.18653/v1/n19-1357>.
- [7] S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *arXiv:1705.07874 [cs, stat]*, Nov. 2017, Available: <https://arxiv.org/abs/1705.07874>
- [8] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, “Human-in-the-loop machine learning: a state of the art,” *Artificial Intelligence Review*, vol. 56, Aug. 2022, doi: <https://doi.org/10.1007/s10462-022-10246-w>.

- [10] “(PDF) Scikit-learn: Machine Learning in Python,” *ResearchGate*, Available: https://www.researchgate.net/publication/51969319_Scikit-learn_Machine_Learning_in_Python
- [11] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” *arXiv.org*, Jun. 16, 2016. <https://arxiv.org/abs/1606.05250>
- [12]. M. T. Ribeiro, S. Singh, and C. Guestrin, ““Why Should I Trust You?”: Explaining the Predictions of Any Classifier,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16*, pp. 1135–1144, Aug. 2016, doi: <https://doi.org/10.1145/2939672.2939778>.
- [13] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv.org*, 2019. <https://arxiv.org/abs/1910.01108>
- [15] A. Vaswani *et al.*, “Attention Is All You Need,” *Cornell University*, Jun. 12, 2017. <https://arxiv.org/abs/1706.03762>